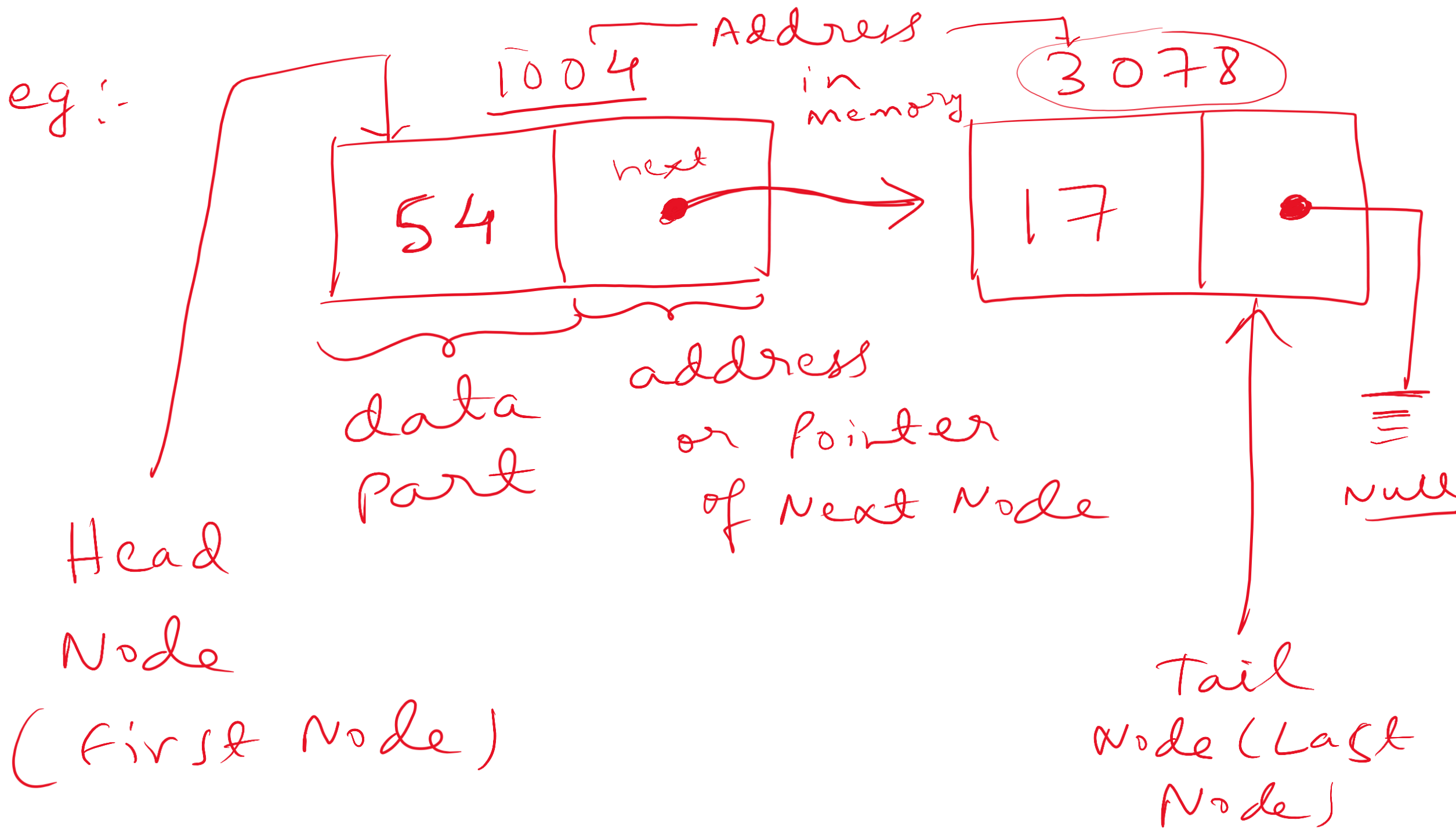
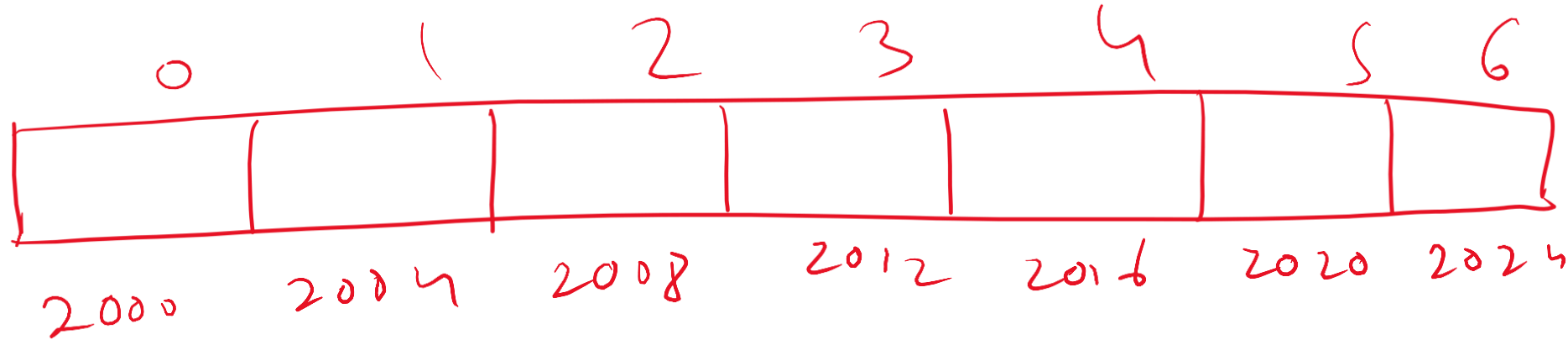


○ Linked List :- It is a linear data structure in which the elements are stored in Nodes and each Node contains the data part and address part (or pointer) to store (or point to) next Node.

eg:-

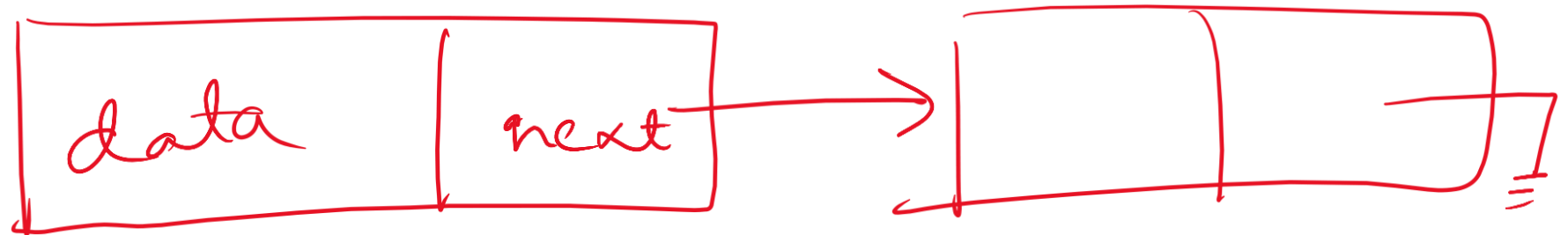


①

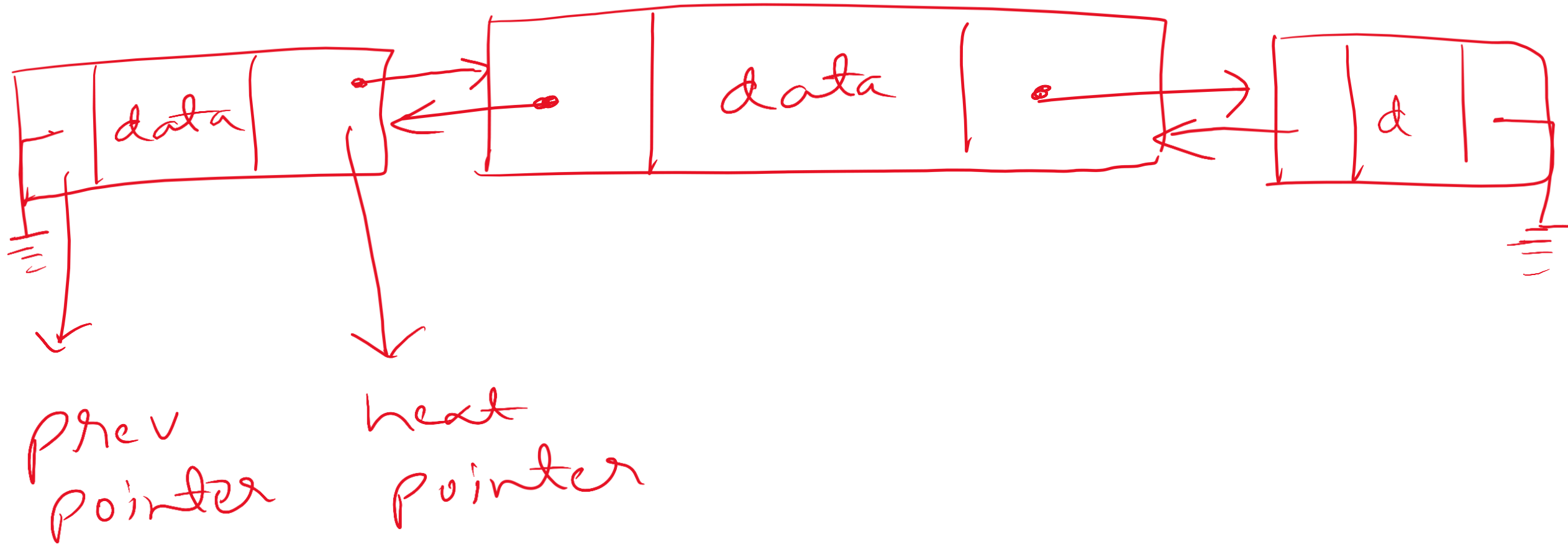


① class (struct) Node
{
 int data; // data part

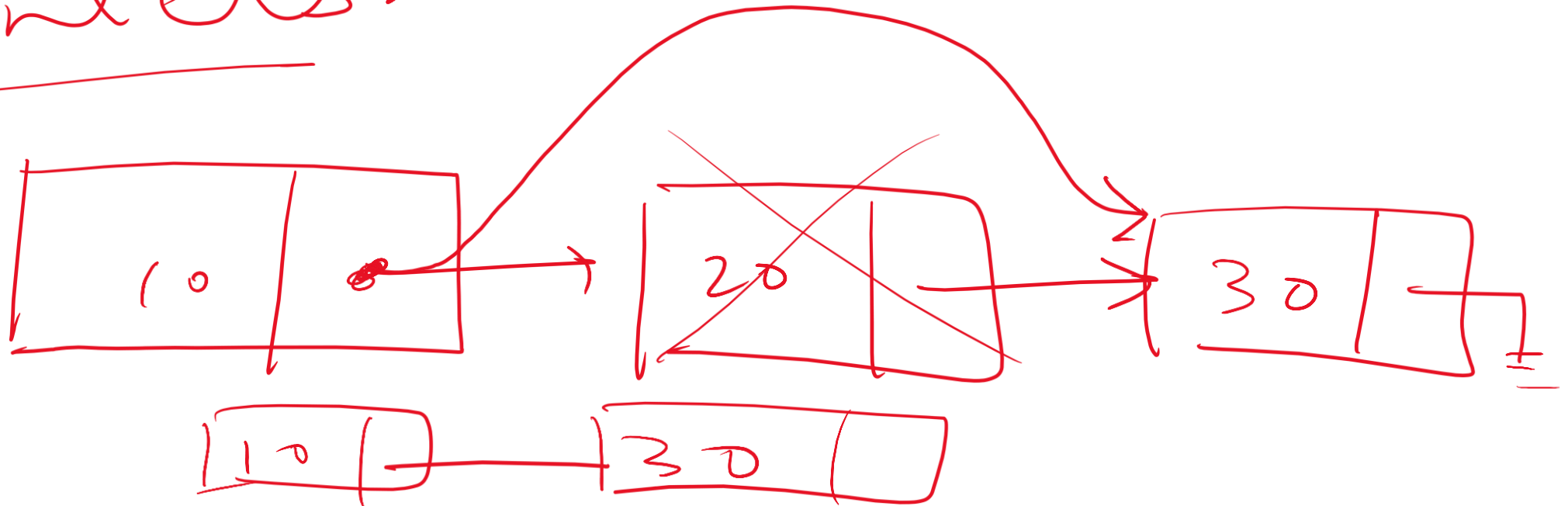
 struct Node * next;
 // pointer
};



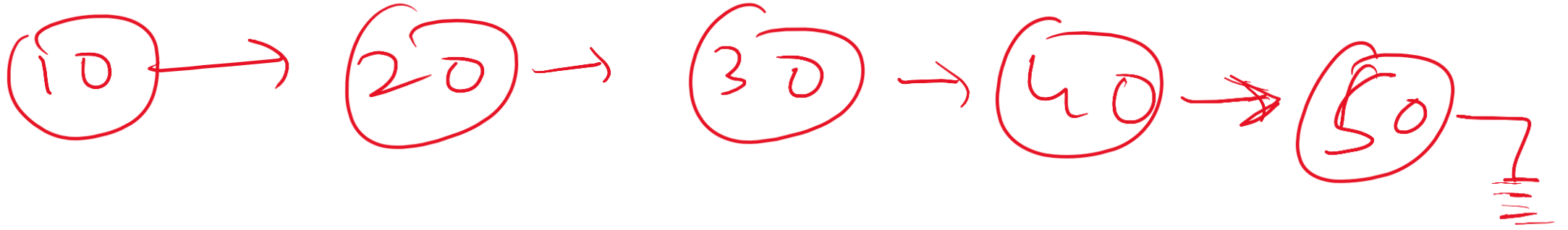
① Doubly Linked List :-



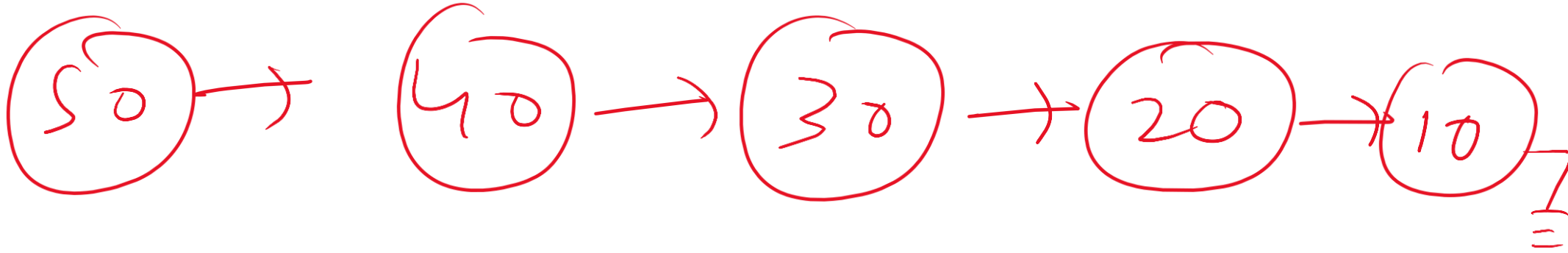
① In Linked List, most of the problems will revolve around manipulation of pointers.



①



②

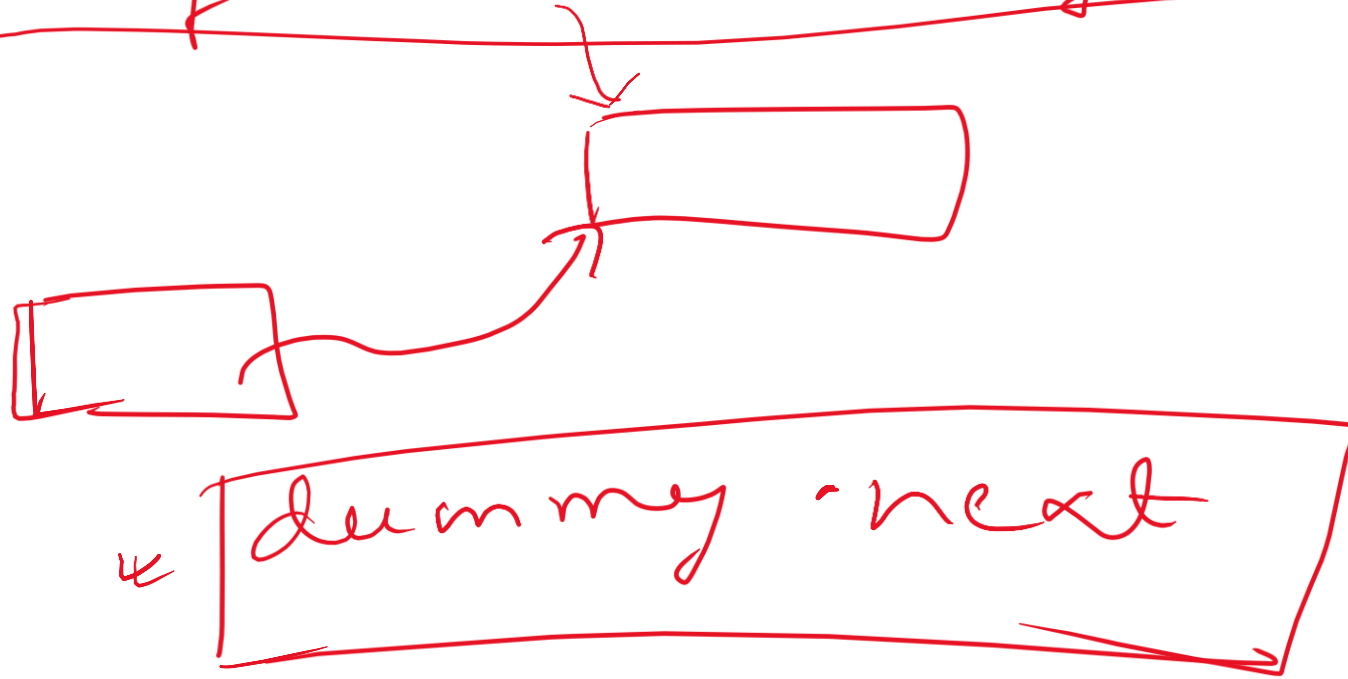


② Two pointer (one will be slow pointer which will move only one Node at a time and other will be fast pointer which will be moving 2 Nodes at a time)

$$\& i = i + 1 \quad i'$$

$$\& i = i + 2, i'$$

① Use of a dummy Node.



② Use a pointer of next, curr and prev.

①

$next = \underline{curr \cdot next};$

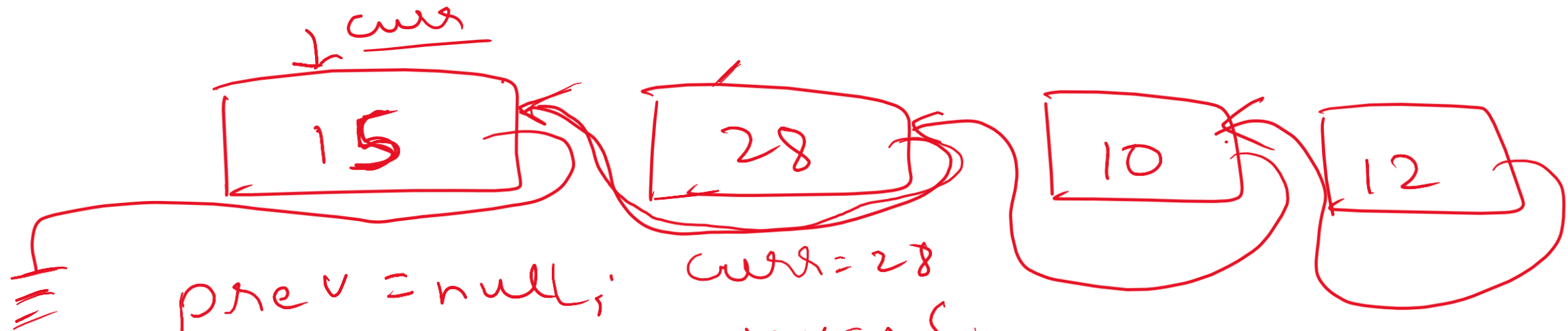
$curr \cdot next = \underline{prev};$

$prev = curr;$

$curr = next;$

Reverse
a
Linked
List

0



$prev = null;$

$curr = 15$

$next = 28$

$curr = 28$

$prev = 15,$

$next_next = curr_next = 12$

$curr_next = prev = null;$

$prev = curr = 28$

$curr = next, = 10$

$\text{prev} = \text{head};$

$\text{curr} = \text{head} \cdot \text{next};$

↳ ②

while ($\text{curr} \neq \text{null}$)

{

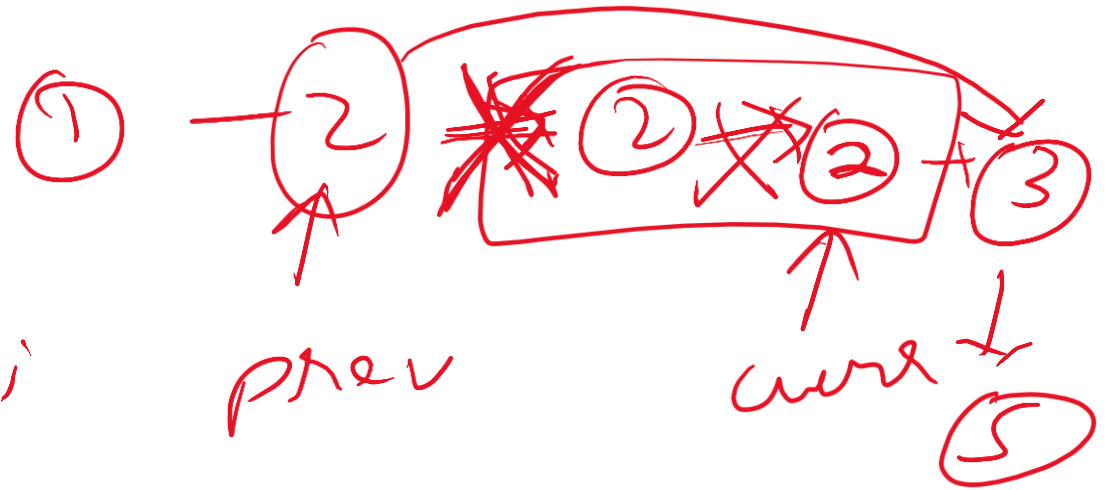
if ($\text{prev} \cdot \text{data} \neq \text{curr} \cdot \text{data}$)

{

$\text{prev} = \text{curr};$

$\text{curr} = \text{curr} \cdot \text{next};$

}



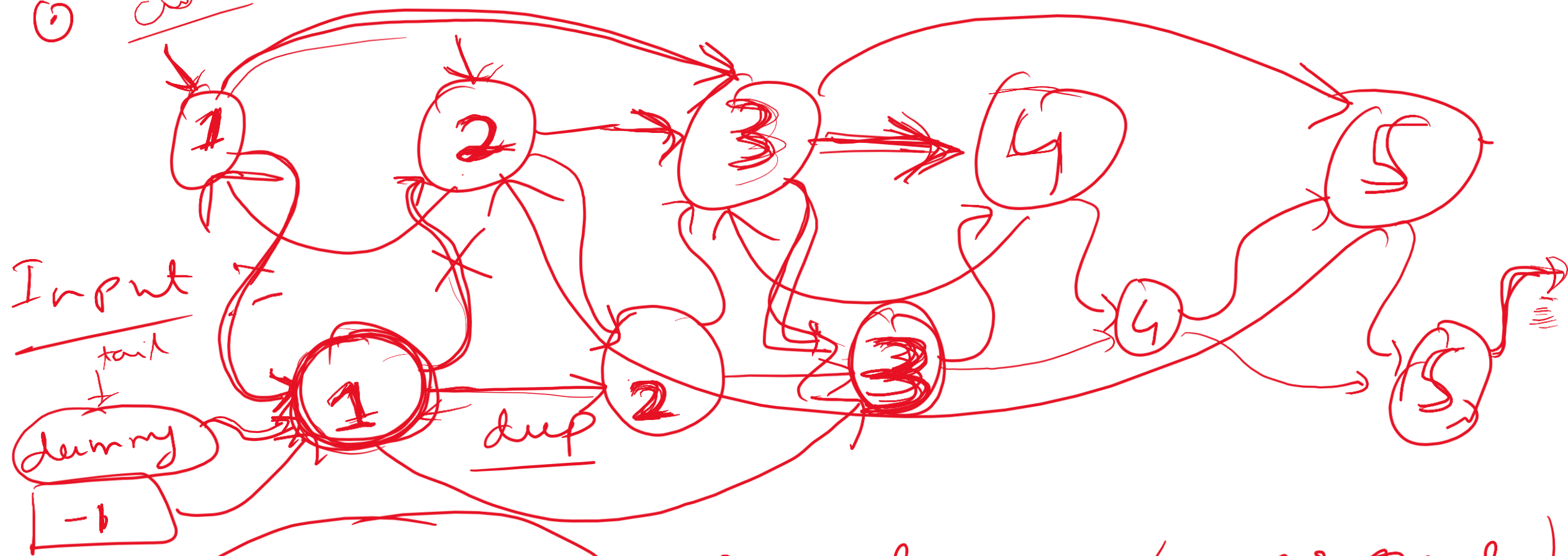
else

{ prev.next = curr.next;

curr = curr.next;

}

0 curr



$\text{curr.next} \cdot \text{random} = \underline{\text{curr.random}}$
 dup $\underline{\text{curr}} = (\text{curr.next}, \text{next}, \text{next}, \text{next})$